

Dynamic Louvain on GPUs with CUDA

B.Tech Project - Final Presentation

Raadhes Chandaluru
Under Prof. Rupesh Nasre

Department of Computer Science and Engineering
Indian Institute of Technology Madras

June 2026

Overview

- 1 Introduction
- 2 The Louvain Algorithm
- 3 GPUs & CUDA
- 4 Dynamic Community Detection
- 5 Implementation, Challenges & Results
- 6 Conclusion

Overview

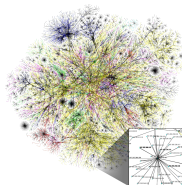
- 1 Introduction
- 2 The Louvain Algorithm
- 3 GPUs & CUDA
- 4 Dynamic Community Detection
- 5 Implementation, Challenges & Results
- 6 Conclusion

Graphs are Everywhere

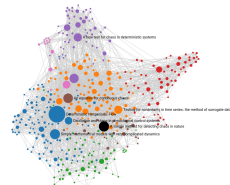
Almost every modern system is a **network** of **vertices** and **edges**:



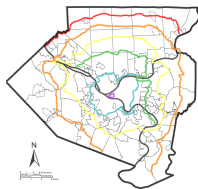
Social networks



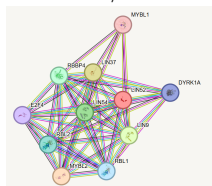
The web / internet



Citation networks

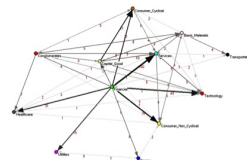


Road networks



Protein interactions

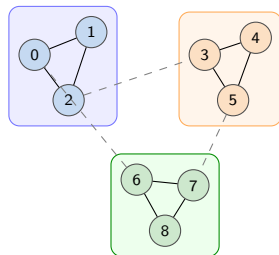
Images: Wikimedia Commons (CC).



Financial networks

Community Detection

- **Community detection** partitions the vertices into groups that are **densely connected inside** and **sparsely connected outside**.
- These groups carry meaning - friend circles, topical web pages, functional protein modules.
- Powers **influence maximisation**, **fraud detection**, **recommendation**, and **protein-function prediction**.



Dense inside, sparse between.

Overview

- 1 Introduction
- 2 The Louvain Algorithm**
- 3 GPUs & CUDA
- 4 Dynamic Community Detection
- 5 Implementation, Challenges & Results
- 6 Conclusion

Modularity - the Objective

Modularity Q scores a partition: observed intra-community edge fraction *minus* what we'd expect at random.

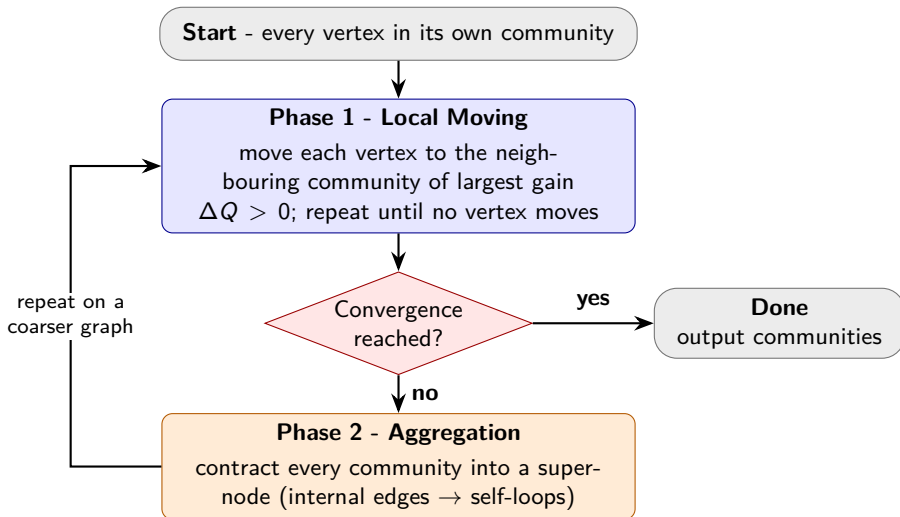
$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad \in [-0.5, 1], \text{ higher is better}$$

Equivalently, as a sum of **per-community** contributions:

$$Q = \sum_c Q_c, \quad Q_c = \frac{\Sigma_{in}(c)}{2m} - \left(\frac{\Sigma_{deg}(c)}{2m} \right)^2$$

- m : total edge weight A_{ij} : edge weight k_i : weighted degree
- $\Sigma_{in}(c)$: edges inside c $\Sigma_{deg}(c)$: total degree of c
- $\delta(c_i, c_j) = 1$ iff i, j in the same community.

The Louvain Method - Two Repeating Phases



Modularity Gain ΔQ

Moving node i from community a to community b :

$$\Delta Q_{i:a \rightarrow b} = \frac{1}{m} \left[(k_{i,b} - k_{i,a}) - \frac{k_i}{2m} (\Sigma_b - \Sigma_a) \right]$$

- $k_{i,x}$: weight of edges from i into community x
- k_i : weighted degree of i Σ_x : total degree of community x

For a node leaving its **own singleton** ($k_{i,a} = 0$, $\Sigma_a = k_i$) this simplifies to

$$\Delta Q_{i \rightarrow b} = \frac{1}{m} \left[k_{i,b} - \frac{k_i \Sigma_{deg}(b)}{2m} \right]$$

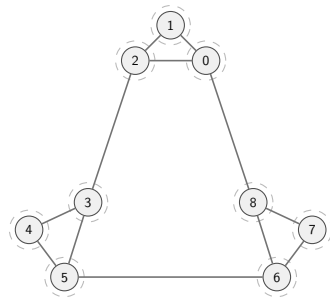
Only **local** quantities - cheap to evaluate per node.

Worked Example (1/6) - Setup ($m = 12$)

Nine nodes: **three triangles** joined in a ring by three bridge edges. Every node starts in its **own** community.

Six bridge nodes have degree 3, three outer nodes degree 2:

$$\begin{aligned} Q_0 &= -\frac{1}{(2m)^2} \sum_i k_i^2 \\ &= -\frac{6 \cdot 3^2 + 3 \cdot 2^2}{24^2} = -\frac{66}{576} \approx -0.115. \end{aligned}$$



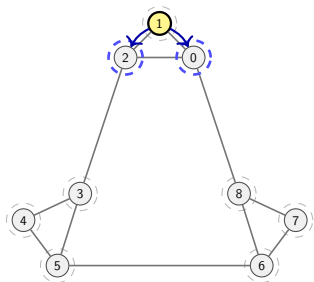
Worked Example (2/6) - Local Moving: Node 1

Every node sits in its **own community** (dashed circles). **Node 1** ($k_1=2$) tests its two neighbours $\{0\}$ and $\{2\}$:

$$\Delta Q_{1 \rightarrow \{0\}} = \frac{1}{12} \left[1 - \frac{2 \cdot 3}{24} \right] = 0.0625$$

$$\Delta Q_{1 \rightarrow \{2\}} = \frac{1}{12} \left[1 - \frac{2 \cdot 3}{24} \right] = 0.0625$$

Both are > 0 and *equal*; the tie is broken arbitrarily, so node 1 joins $\{0\}$.



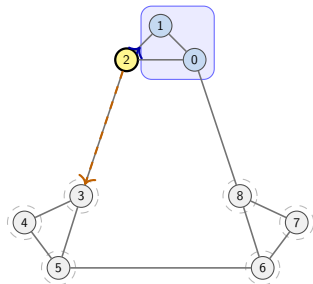
Worked Example (3/6) - Local Moving: Node 2

Now $\{0, 1\}$ is one community. **Node 2** ($k_2=3$) chooses between joining $\{0, 1\}$ (two edges) or the singleton $\{3\}$ (one bridge):

$$\Delta Q_{2 \rightarrow \{0,1\}} = \frac{1}{12} \left[2 - \frac{3 \cdot 5}{24} \right] \approx 0.115$$

$$\Delta Q_{2 \rightarrow \{3\}} = \frac{1}{12} \left[1 - \frac{3 \cdot 3}{24} \right] \approx 0.052$$

The larger gain wins, so node 2 joins $\{0, 1\}$, forming $\{0, 1, 2\}$.

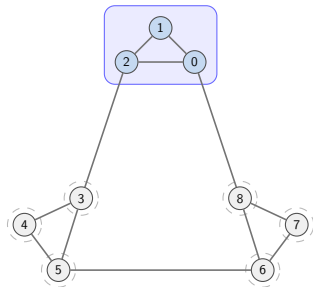


Worked Example (4/6) - Local Moving (continued)

Node 2 joined, so $\{0, 1, 2\}$ - the whole top triangle - is now one community.

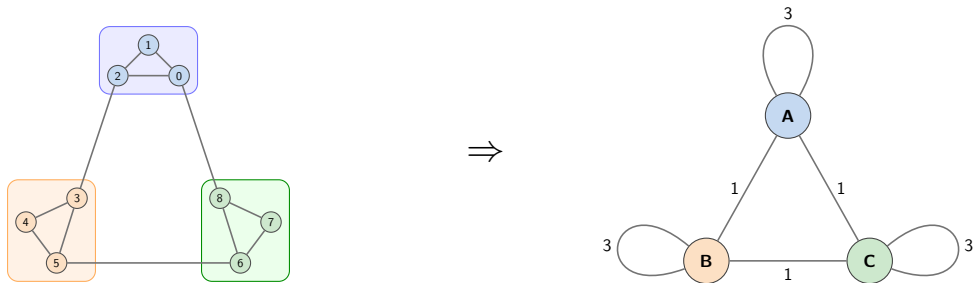
The local-moving phase **keeps going**, visiting the remaining nodes 3–8 the same way: each is pulled in by its strong intra-triangle edges, while the single bridges are too weak to merge triangles.

The pass ends once no node can improve Q .



Worked Example (5/6) - Aggregation

Local moving ends with **three communities**, each a triangle ($\Sigma_{in} = 3$, total degree 8), giving $Q \approx 0.42$. Each community now **contracts into a super-node**:



Three internal edges $\rightarrow$ a **self-loop** (weight 3); each bridge $\rightarrow$ a unit edge.

Worked Example (6/6) - Next Level: No Improvement

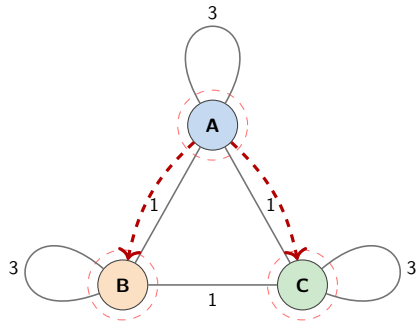
A second local-moving pass starts on the 3-node graph - each super-node is again its **own community** (dashed). **A** tries to move to **B** or **C**:

$$\Delta Q_{A \rightarrow B} = \frac{1}{12} \left[1 - \frac{8 \cdot 8}{24} \right] = -\frac{5}{36} \approx -0.139 < 0$$

(and $\Delta Q_{A \rightarrow C}$ is the same). The heavy self-loops outweigh the single bridges, so **no move improves** Q .

Converged

No vertex moves $\Rightarrow$ the algorithm **stops** with **three communities**, $Q \approx 0.42$.

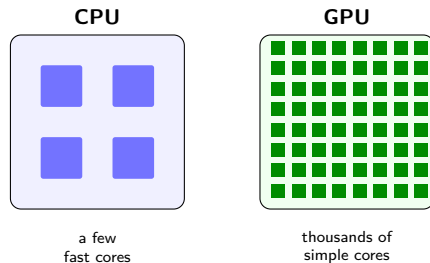


Overview

- 1 Introduction
- 2 The Louvain Algorithm
- 3 GPUs & CUDA**
- 4 Dynamic Community Detection
- 5 Implementation, Challenges & Results
- 6 Conclusion

GPUs: Built for Throughput

- A handful of fast CPU cores vs **thousands of simple GPU cores** running **thousands of threads** at once.
- All that compute is exactly why **we want to run Louvain on the GPU**.



- **CUDA** is NVIDIA's programming model for turning that hardware loose.
- You write a **kernel** - a C/C++ function run by **thousands of threads** at once.
- Launch it with `<<grid, block>>`; here one block of 32 threads, each printing once.

The CUDA "Hello World"

```
__global__ void dkernel() {  
    printf("Hello World\n");  
}  
  
int main() {  
    dim3 grid(1, 1, 1);  
    dim3 block(32, 1, 1);  
    dkernel<<<grid, block>>>();  
    cudaDeviceSynchronize();  
}
```

Overview

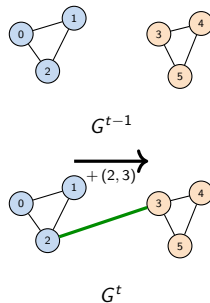
- 1 Introduction
- 2 The Louvain Algorithm
- 3 GPUs & CUDA
- 4 Dynamic Community Detection**
- 5 Implementation, Challenges & Results
- 6 Conclusion

Real Networks Keep Changing

Classic Louvain assumes a **fixed snapshot**. But real graphs **never stop changing** - vertices and edges arrive and disappear all the time:

- new roads added to a city;
- new accounts, new friendships, banned users on social media;
- new papers entering a citation database;
- updated protein-interaction findings.

Each update is a **batch** of edge insertions / deletions, taking $G^{t-1} \rightarrow G^t$.



...and the Graphs are Huge

Researchers already run *Louvain* on exactly these massive, evolving graphs:

- **Twitter / X during the 2022 Ukraine war** - a communication graph of **3.2 M users / 27.4 M edges**; pro/anti factions form and shift over the first four weeks. Communities are detected with **Louvain** (and compared against Leiden).

Śliwa, Kušen, Strembeck - "Comparing Twitter Communities by Louvain & Leiden during the 2022 War in Ukraine", WWW '24 Companion.

- **Opinion communities on Reddit** reorganise over months and years - again tracked using **Louvain** community detection.

Reddit opinion-dynamics study, Information Sciences, 2026.

Millions of edges + a new batch arriving constantly $\Rightarrow$ re-running static Louvain every time is **infeasible**.

The Problem - and the Gap We Fill

- The classic **Louvain** method is built for a **single static snapshot**.
- On a changing graph the usual practice is to **re-run it from scratch** after every update - prohibitive at scale and frequency.
- **Dynamic** variants instead **update incrementally** from the previous partition.
- On **GPUs**, fast static Louvain exists (cuGraph, cuVite, Nido) - but a **dynamic GPU** implementation is almost unexplored.

This project

A from-scratch CUDA **GPU dynamic Louvain**: a corrected static algorithm *plus* three incremental variants (ND, DS, DF), benchmarked against cuGraph, NetworkKit and NetworkX.

The Research Landscape - Where We Sit

	Static	Dynamic
CPU	Blondel '08 (original) Grappolo (Lu '15) NetworkKit / PLM	Naive-Dynamic (Aynaud) Δ -Screening (Zarayeneh) DF-Louvain (Sahu) C-Blondel
GPU	HyDetect, ACLM, Nido cuVite, cuGraph ν -Louvain	almost empty ← this work

Interesting find from the survey: **Leiden** often beats Louvain in quality (cf. the Ukraine study) - a natural future direction.

From Static to Dynamic

Static Louvain is run **once** on the initial graph. After that, updates arrive as **batches** - each batch is a set of edge **deletions and additions**. The two phases are unchanged; dynamic Louvain differs in just **two** ways:

- **Warm start.** Start each node from its community in the *previous* partition (not singletons) - already near-optimal, so few moves are needed.
- **Affected vertices.** Only process the nodes whose community might change; the rest stay put.

The central design choice = the *affected set*

Too small $\Rightarrow$ miss moves, quality drops. Too large $\Rightarrow$ a lot more compute than needed.

Three Ways to Mark the Affected Set

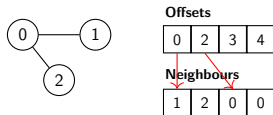
Variant	Affected set	How it is found
ND	every vertex	warm start only; no locality
DS	fixed superset, up front	screen by ΔQ : mark endpoint, its neighbours & the whole target/source community
DF	minimal seed that grows	mark changed endpoints; when a vertex moves, mark its neighbours

Overview

- 1 Introduction
- 2 The Louvain Algorithm
- 3 GPUs & CUDA
- 4 Dynamic Community Detection
- 5 Implementation, Challenges & Results**
- 6 Conclusion

Graph - Compressed Sparse Row (CSR)

- Real graphs are **sparse** ($|E| \ll |V|^2$) - an adjacency matrix is hopeless.
- An offset array + a flat neighbour array; vertex v 's neighbours are the range $[\text{off}[v], \text{off}[v+1])$.
- Memory $O(|V| + |E|)$; coalesced, $O(1)$ access to a neighbour list.



Per-vertex hashtable

- Choosing a vertex's best community needs its **weight to each neighbouring community**.
- Each vertex owns a **private open-addressing hashtable** (keyed by community), sized to its degree - so accumulation needs **no locks**.
- Held in one global `htKey/htVal` pair, split per vertex by a prefix-sum offset array.

Plus length- $|V|$ state arrays: community C , degree K , community totals Σ , and the active/range masks.

The Local-Moving Kernel: Verified Commit Under Lock

One GPU thread per vertex v , in **four steps**:

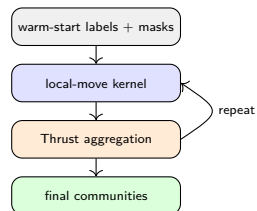
- 1 **Accumulate** - add v 's edge weight to each neighbouring community in its hashtable slice.
- 2 **Optimistic pick** - choose best_c with the largest gain, reading Σ *optimistically* (may be stale).
- 3 **Lock** the **community pair** $(d, \text{best_c})$, lower id first (deadlock-free).
- 4 **Verify & commit** - with both communities frozen, recompute the *exact* ΔQ and move **only if it is still positive**.

Why it is correct

Holding both community locks during verification means no other thread can change either community's membership $\Rightarrow$ **every committed move provably increases modularity** (no stale- Σ moves, no 2-cycle swaps).

One Driver

- A **single shared multi-level driver** runs the **static pass** and **all three dynamic variants** (ND, DS, DF).
- They differ only in two inputs: the **warm-start community labels** and the initial active/range masks.
- Each level alternates the **local-moving kernel** with **Thrust aggregation**, then repeats on the coarser graph until the partition stops improving.



Improvements since UGRC

The UGRC predecessor “*collapsed into a single community . . . likely a synchronization error*” - it did so **sometimes, on large graphs**. We traced and fixed exactly this:

- **Self-loop ΔQ bug.** After aggregation every super-node has a large self-loop; the old gain wrongly counted it $\Rightarrow$ inflated gains $\Rightarrow$ **communities collapsed** from pass 2 onward. *Fix*: skip the self-loop.
- **Float-noise thrashing.** Near-zero equal-and-opposite gains kept passes oscillating. *Fix*: a strict positivity threshold `MOVE_EPS` (10^{-12}).

Result: modularity is now non-decreasing. No collapse.

Challenges

- **Edge-parallel** → **vertex-parallel**. UGRC split the arc array and needed **four spin-locks** per move. This change required some rework.
- **A per-vertex hash table**. I did not consider this initially but saw it in a paper. This is to store weight of edges to each neighbour of a vertex - required for gain calculations.
- **Cooperative kernels** → **per-iteration relaunch**. `grid.sync()` needs all blocks co-resident (≈ 48 on my RTX3050) - can't fit millions of vertices. Now the **host relaunches** the kernel each iteration; the launch boundary *is* the global barrier $\Rightarrow$ scales to any size.

- **Coalesced memory access.** The graph is stored as a **structure-of-arrays**, so the 32 lanes of a warp read **adjacent words** in a single memory transaction - rather than strided reads from an array-of-structures.

An Interesting Find

- The *same* source **hangs on sm_60** (pre-Volta) but runs in **milliseconds on sm_86**. The community-pair locks rely on Volta+ **Independent Thread Scheduling**, where the lanes of a warp make independent progress.

Experimental Setup

Hardware (Google Colab)

- **NVIDIA T4** (15 GB) - most graphs
- **NVIDIA A100** (40 GB) - the two largest

Baselines

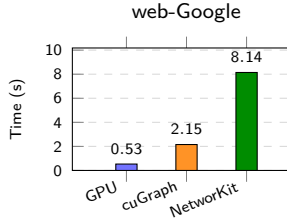
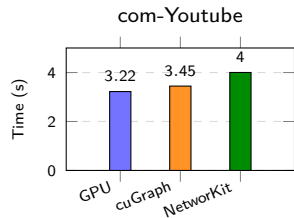
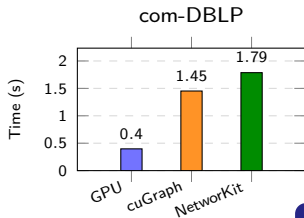
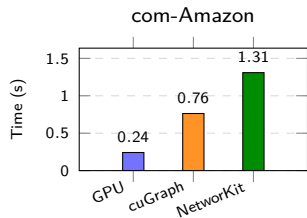
- **NetworkX** - single-thread CPU
- **NetworKit** - parallel CPU (PLM)
- **cuGraph** - GPU (RAPIDS)

Graphs (common format; undirected, unit weight)

- 2 classic (karate, dolphins) - correctness
- **12 SNAP static**: 4 K–3 M vertices, 88 K–**117 M** edges
- **4 SNAP temporal**: 80% initial + $5 \times 4\%$ insertion batches
- synthetic churn on com-DBLP (quality)

Metrics: modularity, # communities, wall-clock time.

Static - Performance



- All four implementations reach **essentially the same modularity**.
- These are four of the largest SNAP graphs.

Headline (six largest SNAP)

geo-mean **1.74** $\times$ vs cuGraph, **2.18** $\times$ vs NetworkKit.

web-Google (4.3 M edges): 0.53 s = **4** $\times$ / **15** $\times$ / **450** $\times$ vs cuGraph / NetworkKit / NetworkX.

Static - Why hubs are reducing performance

Graph	$ V $	$ E $	Avg	Max	vs cuGraph	vs NetworKit
com-DBLP	317,080	1,049,866	6.6	343	3.66×	4.50×
com-Amazon	334,863	925,872	5.5	549	3.15×	5.41×
web-Google	875,713	4,322,051	9.9	6,332	4.05×	15.31×
com-Youtube	1,134,890	2,987,624	5.3	28,754	1.07×	1.24×
com-LiveJournal	3,997,962	34,681,189	17.4	14,815	1.03×	0.81×
com-Orkut	3,072,441	117,185,083	76.3	33,313	0.54×	0.29×

- The split tracks **maximum degree**: bounded-degree graphs win big; **hub-dominated** social graphs collapse the advantage.
- **Why**: a *vertex-parallel* warp stalls when 31 lanes finish and one holds a degree-33,000 hub - classic load imbalance.

Dynamic - Incremental Updates are Far Faster

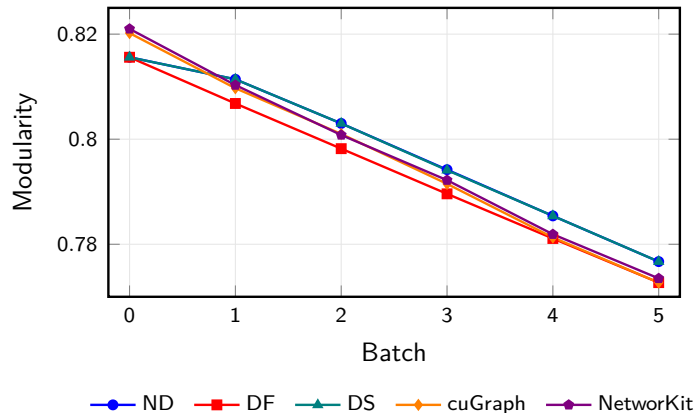
Avg. per-batch time (ms)	ND	DF	DS	cuGraph	NetworkKit	NetworkX
CollegeMsg	1.2	0.4	1.2	64.8	3.3	312.5
sx-mathoverflow	50.0	23.6	49.6	264.5	73.3	6,176.0
sx-askubuntu	68.6	49.8	109.2	439.3	288.6	32,476.4
sx-superuser	95.6	66.4	96.2	682.0	434.4	45,998.0

- Baselines must **re-cluster the whole graph** every batch; our variants reprocess only the **affected vertices** from the warm start.
- **DF** touches the smallest set $\Rightarrow$ fastest.

Headline (DF, 4% batches, geo-mean)

20.14 $\times$ vs cuGraph, **5.58 $\times$** vs NetworkKit,
hundreds $\times$ vs NetworkX.

Dynamic - Quality is Preserved



com-DBLP, synthetic churn (**60% insert** / **40% delete**), five cumulative batches.

- All five curves stay **within** 0.01 at every batch.
- Holds even for **20%-of- $|E|$** batches.
- Incremental variants **track full re-clustering** - the degradation is the *perturbed graph's*, not the method's.

Overview

- 1 Introduction
- 2 The Louvain Algorithm
- 3 GPUs & CUDA
- 4 Dynamic Community Detection
- 5 Implementation, Challenges & Results
- 6 Conclusion**

Conclusion

- Built, from scratch in CUDA, a **GPU dynamic Louvain**: a corrected static algorithm + three incremental variants (ND, DS, DF) on **one shared driver**.
- Resolved the UGRC correctness failures - moves are now **modularity non-decreasing**.
- **Static**: reference-quality partitions, with a **speed-up over cuGraph and NetworkKit** on large graphs; honest limit on hub-dominated graphs.
- **Dynamic**: **20× vs cuGraph**, several× vs NetworkKit, hundreds× vs NetworkX - at **little to no quality cost**.
- To our knowledge, the **only public GPU dynamic Louvain** apart from StarPlat.

Algorithmic

- Modify the algorithm to be **more GPU-specific** (not just a CPU port).
- Explicit **vertex** add/delete.
- A dynamic GPU **Leiden**.

Evaluation

- **NMI / ARI** vs ground truth; modularity density. (I do not know what each is in detail)
- Direct **DF-Louvain** (OpenMP) baseline.
- Just keep profiling.
- Benchmark: Billions of edges **uk-2007**; weighted & real-deletion temporal graphs.

Publication

- Parallel-computing venues: IJPP, JPDC, IEEE TPDS, ICPP.

Q/A?

Code: `github.com/KlowdfurrRad/dynamic_louvain_cuda__BTP`

LLMs were used in slide generation.